Практическая работа №3
Форматы данных

Предмет: Инструментальные средства разработки программного обеспечения
Выполнил: Богданов Роман Романович. Группа: ИС-41

Цель работы

Сравнить JSON, XML, YAML, MessagePack, CBOR и Protobuf по размеру, скорости и читаемости. Передать профиль пользователя по TCP и проверить чтение старых данных новой схемой Protobuf.

1. Ответы на вопросы

Текстовые форматы хранят данные в виде текста, поэтому их удобно читать и проверять вручную. Бинарные форматы используют специальные коды и компактное представление значений; для просмотра требуется программа-декодер. Размер и скорость зависят от структуры данных и реализации библиотеки.

Схемная эволюция - изменение структуры сообщения с сохранением совместимости версий. В Protobuf новые поля добавляют с новыми номерами, а старые номера не переиспользуют. Отсутствующее обычное строковое поле proto3 читается как пустая строка.

2. Подготовка данных и окружения

Создан каталог date_format со всеми скриптами из задания, исходной и сгенерированной схемой. Использован профиль из условия: id=42, Анна Смирнова, 22 года, студент, три курса, адрес Москва, индекс 101000. Запуск выполнен в Python 3.12.14. Версии библиотек закреплены в requirements.txt.

Таблица 1. Текстовые форматы

Формат	Размер, байт	Читаемость (1-5)	Комментарии
JSON	253	5	Нет
XML	283	3	Да: <!-- ... -->
YAML	182	5	Да: # ...

Здесь JSON записан с indent=2, как в step_json.py. В сводном сравнении используется JSON без отступов, как в run_all.py из задания. Поэтому размеры JSON в двух таблицах различаются; это результат форматирования, а не изменения данных.

3. Сравнение всех форматов

Формат	Байт	Читается человеком	Нужна схема
json	211	Да	Нет
xml	283	Да	Нет
yaml	182	Да	Нет
msgpack	157	Нет	Нет
cbor	159	Нет	Нет
protobuf	117	Нет	Да

Размеры относятся только к полезной нагрузке UTF-8/бинарного сообщения. Служебный заголовок нашего TCP-протокола занимает дополнительно 5 байт: 1 байт формата и 4 байта длины. Сетевые заголовки TCP/IP в таблицу не включены.

Измерение скорости

Формат	Сериализация, мкс	Десериализация, мкс
json	2.587	2.462
xml	25.211	13.699
yaml	203.724	338.312
msgpack	1.012	1.082
cbor	2.788	2.745
protobuf	2.942	3.911

Для каждой операции взята медиана пяти серий по 1000 повторений timeit. Файловая запись, печать и передача по сети в замер не входят. Для Protobuf учитывается преобразование Python-словаря в сообщение и обратно. Это результаты одного профиля на данном окружении, а не универсальный рейтинг форматов.

Проверка данных

Все шесть форматов успешно восстановили исходный профиль. Проверяются все поля, включая логический тип is_student, список courses и вложенный address. В XML типы id, age и is_student восстанавливаются явно, поскольку простой XML-текст сам не хранит типы Python.

4. Вывод скриптов JSON и Protobuf

Ниже и далее изображения сформированы из журналов фактических запусков программы. Это сохранённый консольный вывод, а не снимки рабочего стола; исходные текстовые журналы приложены в logs/.

Фактический вывод: JSON и Protobuf

```
$ python step_json.py
=== JSON ===
{
  "id": 42,
  "name": "Анна Смирнова",
  "age": 22,
  "is_student": true,
  "courses": [
    "Математика",
    "Физика",
    "Информатика"
  ],
  "address": {
    "city": "Москва",
    "zip": "101000"
  }
}
Размер: 253 байт
Десериализация успешна: Анна Смирнова
Все поля совпадают с исходными: True

$ python step_protobuf.py
=== PROTOBUF ===
HEX: 082a1219d090d0bdd0bdd0b020d0a1d0bcd0b8d180d0bdd0bed0b2d0b0181620012a14d09cd0b0d182d0b5d0bcd0b0d182
d0b8d0bad0b02a0cd0a4d0b8d0b7d0b8d0bad0b02a16d098d0bdd184d0bed180d0bcd0b0d182d0b8d0bad0b032160a0cd09cd0b
ed181d0bad0b2d0b01206313031303030
Размер: 117 байт
Десериализация успешна: Анна Смирнова
Все поля совпадают с исходными: True
```

5. Передача данных по TCP

Клиент и сервер проверены отдельно для JSON, MessagePack, CBOR и Protobuf. Сервер принимает формат и длину, читает сообщение полностью, декодирует его, сохраняет в received/ и отправляет подтверждение OK. Для проверок использованы свободные локальные порты; обычный запуск использует 127.0.0.1:9999.

TCP передаёт поток байтов: один recv() может вернуть только часть заголовка. Поэтому функция recv_exact() дочитывает нужное число байт и сообщает об обрыве соединения. Дополнительно проверены разбиение заголовка на байты и преждевременное закрытие соединения.

Фактический вывод сервера и клиента: TCP

```
$ python server.py
Сервер слушает 127.0.0.1:50093...
=== Получено 211 байт, формат: json ===
HEX: 7b226964223a2034322c20226e616d65223a2022d090d0bdd0bdd0b020d0a1d0bcd0b8d180d0bdd0bed0b2d0b0222c2022
616765223a2032322c202269735f73747564656e74223a20747275652c2022636f7572736573223a205b22d09cd0b0d182d0b5d
0bcd0b0d182d0b8d0bad0b0222c2022d0a4d0b8d0b7d0b8d0bad0b0222c2022d098d0bdd184d0bed180d0bcd0b0d182d0b8d0ba
d0b0225d2c20226164647265373223a207b2263697479223a2022d09cd0bed181d0bad0b2d0b0222c20227a6970223a2022313
031303030227d7d
TEXT: {"id": 42, "name": "Анна Смирнова", "age": 22, "is_student": true, "courses": ["Математика",
"Физика", "Информатика"], "address": {"city": "Москва", "zip": "101000"}}
DATA: {'id': 42, 'name': 'Анна Смирнова', 'age': 22, 'is_student': True, 'courses': ['Математика',
'Физика', 'Информатика'], 'address': {'city': 'Москва', 'zip': '101000'}}
[СОХРАНЕНО] received/message_json.json (211 байт)

$ python client.py --format json
Отправка в формате json, размер: 211 байт
Отправлено. Сервер подтвердил сохранение.

$ python server.py
Сервер слушает 127.0.0.1:60003...
=== Получено 117 байт, формат: protobuf ===
HEX: 082a1219d090d0bdd0bdd0b020d0a1d0bcd0b8d180d0bdd0bed0b2d0b0181620012a14d09cd0b0d182d0b5d0bcd0b0d182
d0b8d0bad0b02a0cd0a4d0b8d0b7d0b8d0bad0b02a16d098d0bdd184d0bed180d0bcd0b0d182d0b8d0bad0b032160a0cd09cd0b
ed181d0bad0b2d0b01206313031303030
TEXT: <не текст, бинарные данные>
DATA: {'id': 42, 'name': 'Анна Смирнова', 'age': 22, 'is_student': True, 'courses': ['Математика',
'Физика', 'Информатика'], 'address': {'city': 'Москва', 'zip': '101000'}}
[СОХРАНЕНО] received/message_protobuf.pb (117 байт)

$ python client.py --format protobuf
Отправка в формате protobuf, размер: 117 байт
Отправлено. Сервер подтвердил сохранение.
```

6. Сохранённые файлы

В received/ сохранены сообщения всех шести форматов и сводный report.json. JSON открывается как UTF-8 текст, а Protobuf и MessagePack в этом примере не декодируются как обычный UTF-8 текст. Для бинарных файлов показано шестнадцатеричное представление.

Сохранённые файлы и просмотр содержимого

```
received/
  message_cbor.cbor             159 байт
  message_json.json             211 байт
  message_msgpack.msgpack       157 байт
  message_protobuf.pb           117 байт
  message_xml.xml               283 байт
  message_yaml.yaml             182 байт
  report.json                   994 байт

message_json.json: просмотр как UTF-8
{"id": 42, "name": "Анна Смирнова", "age": 22, "is_student": true, "courses": ["Математика", "Физика",
"Информатика"], "address": {"city": "Москва", "zip": "101000"}}

message_protobuf.pb: двоичное содержимое (HEX)
082a1219d090d0bdd0bdd0b020d0a1d0bcd0b8d180d0bdd0bed0b2d0b0181620012a14d09cd0b0d182d0b5d0bcd0b0d182d0b8d
0bad0b02a0cd0a4d0b8d0b7d0b8d0bad0b02a16d098d0bdd184d0bed180d0bcd0b0d182d0b8d0bad0b032160a0cd09cd0bed181
d0bad0b2d0b01206313031303030

message_msgpack.msgpack: двоичное содержимое (HEX)
86a269642aa46e616d65b9d090d0bdd0bdd0b020d0a1d0bcd0b8d180d0bdd0bed0b2d0b0a361676516aa69735f73747564656e7
4c3a7636f757273657393b4d09cd0b0d182d0b5d0bcd0b0d182d0b8d0bad0b0acd0a4d0b8d0b7d0b8d0bad0b0b6d098d0bdd184
d0bed180d0bcd0b0d182d0b8d0bad0b0a76164647265737382a463697479acd09cd0bed181d0bad0b2d0b0a37a6970a63130313
03030
UTF-8: не декодируется как обычный текст.
```

7. Проверка схемной эволюции

Сначала скомпилирована старая схема legacy/profile.proto без email и сохранён message_protobuf.pb. Затем скомпилирована новая схема с полем string email = 7. Новый код прочитал старый файл: имя и остальные данные сохранились, значение email равно пустой строке. Дополнительно проверено совпадение байтов старого сообщения до и после остальных запусков.

Проверка новой схемы на старом сообщении

```
Старая схема: поле email отсутствует; сохранено 117 байт

$ python check_evolution.py
Новая схема: добавлено string email = 7;
Старый файл: message_protobuf.pb
Имя: Анна Смирнова
Email (новое поле): ''
Остальные поля совпадают: True
Новый код успешно прочитал старые данные без email.

Python: 3.12.14
Замер: медиана 5 серий по 1000 операций; время в мкс/операцию.
Формат           Байт  Запись, мкс  Чтение, мкс   Читаем?
json              211        2.587        2.462        да
xml               283       25.211       13.699        да
yaml              182      203.724      338.312        да
msgpack           157        1.012        1.082       нет
cbor              159        2.788        2.745       нет
protobuf          117        2.942        3.911       нет
Все 6 форматов: исходные данные восстановлены без потерь.
Размеры указаны без TCP-заголовка приложения (5 байт).
```

8. Выводы

Protobuf оказался самым компактным: 117 байт вместо 211 байт у JSON без отступов. Он хранит номера полей вместо их полных имён и использует компактное кодирование значений. MessagePack занял 157 байт: имена ключей сохраняются, но значения и структура кодируются бинарно.

Вручную удобнее всего читать JSON с отступами; YAML также читается легко и позволяет писать комментарии. XML в этом примере более громоздкий из-за открывающих и закрывающих тегов. Самые быстрые операции в данном замере получились у MessagePack; результат зависит от библиотек и данных.

JSON сам не задаёт правила схемной эволюции, но совместимость можно организовать на уровне приложения: игнорировать неизвестные поля и задавать значения отсутствующих. Protobuf предоставляет такие правила через схему, постоянные номера полей и значения по умолчанию. Проверка показала, что добавление email с новым номером не мешает чтению старого сообщения.